EXAM-DAY CHEAT SHEET - Python (print this, 4 pages)

Version 2: now covers EVERY item on the instructor's practical-assessment checklist. The code blocks form ONE continuous, tested script (pandas -> merge -> charts -> SQLite -> CSV export -> PyTorch regression). Use each block as a set of INDEPENDENT operations: copy only the lines you need and change the names. Explanations: Topics/T1-T9 and Paper_Solutions/P1.

Before you type: exact names? (file, DB, table, columns) - look at the data: `shape`, `head()`, `isna().sum()`, `duplicated().sum()`, `dtypes` - run after every step - print every answer.

A. Imports and loading

```
import sqlite3, os, csv
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import torch, torch.nn as nn, torch.optim as optim

df = pd.read_csv("sales_data.csv")                # load CSV
print(df.shape)                                   # (rows, columns)
print(df.head()); print(df.dtypes); print(df.isna().sum())
```

B. Duplicates, cleaning, merge, filter, save

```
print(df.duplicated().sum())                      # how many repeated rows
print(df[df.duplicated(keep=False)].head(4))      # SEE them (all copies)
before = df.shape
df = df.drop_duplicates()                         # keep first copy
# df.drop_duplicates(subset=["Date", "Product"], keep="first")  # by columns
print("shape", before, "->", df.shape)

df["Product"] = df["Product"].fillna("Unknown")                  # text blank
df["Price"] = df["Price"].fillna(df["Total"] / df["Quantity"])   # recover
df["Price"] = df["Price"].fillna(0)                              # fallback
df["Total"] = df["Quantity"] * df["Price"]                       # fix Total
df["Date"] = pd.to_datetime(df["Date"], dayfirst=True)           # dd/mm/yyyy
# other fills: .fillna(df["x"].mean())  .median()  .mode()[0]   dropna(subset=[..])

# MERGE two DataFrames on a common column
cats = pd.DataFrame({"Product": ["Chocolate", "Biscuits", "Sweets", "Water",
                                 "Orange Juice"],
                     "Category": ["Snack", "Snack", "Snack", "Drink", "Drink"]})
merged = df.merge(cats, on="Product", how="left")   # inner | left | right | outer
print(df.shape, cats.shape, merged.shape)           # CHECK rows did not blow up
# different key names: a.merge(b, left_on="id", right_on="cust_id")
# see unmatched rows: how="outer", indicator=True  -> column _merge

# SELECT / FILTER
d23 = merged[merged["Date"].dt.year == 2023]                      # one condition
big = merged[(merged["Total"] > 100) & (merged["Category"] == "Snack")]  # & | ~  + brackets
two_cols = merged[["Product", "Total"]]                           # choose columns
top = merged.sort_values("Total", ascending=False).head(5)        # sort + first 5
# merged.loc[merged["Total"] > 100, ["Date", "Total"]]    isin([..])   between(a, b)

# SAVE to CSV
big.to_csv("big_sales.csv", index=False)          # index=False: no 0,1,2 column
print(pd.read_csv("big_sales.csv").shape)         # read it back to check
```

C. Matplotlib (every plot: NEW figure + title + xlabel + ylabel)

```
qty = merged.groupby("Product")["Quantity"].sum()                       # pile + add
month = d23.groupby(d23["Date"].dt.month)["Total"].sum()                # 2023 by month

plt.figure(figsize=(8, 5))                          # NEW paper for EVERY chart
plt.bar(qty.index, qty.values)                      # BAR
plt.title("Quantity per product"); plt.xlabel("Product"); plt.ylabel("Quantity")
plt.xticks(rotation=45); plt.tight_layout(); plt.show()

plt.figure(); plt.plot(month.index, month.values, marker="o")           # LINE
plt.title("Sales 2023"); plt.xlabel("Month"); plt.ylabel("Total"); plt.show()

plt.figure(); plt.scatter(merged["Quantity"], merged["Total"])          # SCATTER
plt.title("Quantity vs Total"); plt.xlabel("Quantity"); plt.ylabel("Total"); plt.show()

names = list(qty.index)                                                 # BOX PLOT
groups = [merged.loc[merged["Product"] == p, "Total"] for p in names]   # 1 list per box
plt.figure(); plt.boxplot(groups)                   # box = middle 50%, line = median
plt.xticks(range(1, len(names) + 1), names, rotation=45)   # name the boxes (works in every version)
plt.title("Total by product"); plt.xlabel("Product"); plt.ylabel("Total"); plt.tight_layout(); plt.show()
# one-liner: merged.boxplot(column="Total", by="Product")      histogram: plt.hist(v, bins=5)
# pie: plt.pie(v, labels=l, autopct="%1.1f%%")     save instead of show: plt.savefig("a.png")
```

D. SQLite: DataFrame -> table, SELECT/WHERE/ORDER BY, UPDATE, export to CSV

```
conn = sqlite3.connect("SalesDB.db"); cur = conn.cursor()               # connect
cur.execute("""CREATE TABLE IF NOT EXISTS Sales (
                   Date TEXT, Product TEXT, Quantity INTEGER,
                   Price REAL, Total REAL,
                   UNIQUE (Date, Product, Quantity, Price, Total))""")
out = df.copy(); out["Date"] = out["Date"].dt.strftime("%Y-%m-%d")      # dates -> text
for _, r in out.iterrows():                                             # insert, no duplicates
    cur.execute("INSERT OR IGNORE INTO Sales VALUES (?, ?, ?, ?, ?)",
                (r["Date"], r["Product"], int(r["Quantity"]),
                 float(r["Price"]), float(r["Total"])))
conn.commit()                                       # save! (no commit = lost)

# store a whole DataFrame as a table (one line)
out.to_sql("SalesCopy", conn, if_exists="replace", index=False)         # fail | replace | append

# SELECT + WHERE + ORDER BY with ? parameters
cur.execute("""SELECT Product, SUM(Quantity) AS q FROM Sales
               WHERE strftime('%Y', Date) = ?
               GROUP BY Product ORDER BY q DESC""", ("2023",))
for product, q in cur.fetchall():
    print(product, q)
cur.execute("SELECT * FROM Sales WHERE Product = ? AND Total > ? ORDER BY Date DESC",
            ("Chocolate", 40))                      # ONE value needs a comma: ("x",)
rows = cur.fetchall()
cols = [c[0] for c in cur.description]              # column names - grab NOW (after an
                                                    # UPDATE, description is None)
# UPDATE (always WHERE! check rowcount, then commit)
cur.execute("UPDATE SalesCopy SET Price = ? WHERE Product = ?", (11.0, "Chocolate"))
print(cur.rowcount, "rows updated")                 # how many rows changed
conn.commit()
# arithmetic: UPDATE T SET Price = Price * 1.1 WHERE ...     DELETE FROM T WHERE ...

# EXPORT query results to CSV (two ways)
pd.DataFrame(rows, columns=cols).to_csv("query_result.csv", index=False)
pd.read_sql_query("SELECT * FROM Sales WHERE Product = ?", conn,
                  params=("Water",)).to_csv("water.csv", index=False)
conn.close()
```

E. CSV file: create-or-append, input loop, numpy summary

```
FILE = "heights.csv"
if not os.path.exists(FILE):
    with open(FILE, "w", newline="") as f:
        csv.writer(f).writerow(["Name", "Height"])
    print("heights.csv created")
else:
    print("heights.csv exists - appending")

for name, h in [("Amy", "165"), ("Ben", "172"), ("q", "")]:   # replace with input()
    if name.lower() == "q":                    # name = input("Name (q to quit): ")
        break
    try:
        height = int(h)                        # h = input("Height (cm): ")
    except ValueError:
        print("Height must be a whole number"); continue
    with open(FILE, "a", newline="") as f:     # a = append
        csv.writer(f).writerow([name, height])

h = pd.read_csv(FILE)["Height"].to_numpy()
text = (f"Records: {len(h)}\nAverage: {np.mean(h):.1f}\n"
        f"Min: {np.min(h)}\nMax: {np.max(h)}\n")
print(text)
with open("height_summary.txt", "w") as f:
    f.write(text)
plt.figure(); plt.hist(h, bins=5)
plt.title("Height Distribution"); plt.xlabel("Height (cm)"); plt.ylabel("Frequency")
plt.show()
```

F. PyTorch REGRESSION (the instructor's recipe: y = X @ w + b + noise)

```
torch.manual_seed(42)                          # FIRST - before data AND before the model
N, F = 200, 3                                  # 200 samples, 3 features
X = torch.randn(N, F)                          # X shape (N, F)
w = torch.tensor([[2.0], [-1.0], [0.5]])       # true weights, shape (F, 1)
b = 0.7                                        # true bias
y = X @ w + b + 0.1 * torch.randn(N, 1)        # (N,F)@(F,1) -> (N,1)  + noise
print(X.shape, y.shape)                        # torch.Size([200, 3]) torch.Size([200, 1])

class RegNet(nn.Module):
    def __init__(self):
        super().__init__()                     # FIRST line of __init__, always
        self.fc1 = nn.Linear(F, 8)             # F inputs -> 8 hidden neurons
        self.fc2 = nn.Linear(8, 1)             # 8 -> 1 output number
    def forward(self, x):
        return self.fc2(torch.relu(self.fc1(x)))   # NO sigmoid: output is any real number

model = RegNet()
criterion = nn.MSELoss()                       # continuous target -> MSE
optimizer = optim.Adam(model.parameters(), lr=0.01)    # or optim.SGD(..., lr=0.01)
losses = []
for epoch in range(200):
    optimizer.zero_grad()                      # 1 wipe old gradients
    loss = criterion(model(X), y)              # 2+3 forward pass, loss
    loss.backward()                            # 4 backpropagation
    optimizer.step()                           # 5 update weights
    losses.append(loss.item())                 # record a plain number
print(f"loss {losses[0]:.3f} -> {losses[-1]:.4f}")

plt.figure(); plt.plot(losses)
plt.title("Training loss"); plt.xlabel("Epoch"); plt.ylabel("MSE loss"); plt.show()
with torch.no_grad():                          # check: predictions vs truth
    print(torch.cat([model(X[:3]), y[:3]], dim=1))
# classification instead: end with torch.sigmoid, loss nn.BCELoss(), y = 0./1. shape (N,1)
# 2024 paper: Linear(10,5) relu Linear(5,3) relu Linear(3,1) sigmoid, MSELoss, SGD lr=0.01
```

G. Python reminders

```
f"{x:.2f}" 2 decimals    {x:>8} right-align   {name:<10} left-align
int(input("n: "))        try: ... except ValueError: ...      while True: ... break
for i, v in enumerate(lst):      dict.get(k, 0)      sorted(d.items(), key=lambda kv: kv[1], reverse=True)
class A: def __init__(self, x): self.x = x      def __str__(self): return f"A({self.x})"
and / or / not      == (compare) vs = (assign)       lst[1:4]  s[::-1]   len(x)
```

H. Error -> cause -> fix

You see	Cause	Fix
`ValueError: time data ... doesn't match`	day/month order	`dayfirst=True` or `format=`
`Can only use .dt accessor with datetimelike values`	Date still text	convert with `to_datetime` first
`KeyError: 'zzz'` (merge or select)	wrong column name / spaces	`print(df.columns)`
`ValueError: You are trying to merge on object/str and int64 columns`	key types differ	`a["k"] = a["k"].astype(int)`
merged table has far MORE rows than expected	duplicate keys on both sides	`drop_duplicates` on the key first, compare `shape`
`ValueError: x and y must be the same size`	scatter lists differ in length	check `len()` of both
`OperationalError: no such table` / `no such column: zzz`	queried before CREATE / typo / wrong folder	create first; check names and cwd
`UNIQUE constraint failed`	duplicate with plain INSERT	`INSERT OR IGNORE`
`ProgrammingError: Incorrect number of bindings supplied`	`?` count != tuple length, or `("x")` instead of `("x",)`	match counts; add the comma
BLOB junk / `Error binding parameter`	numpy int / Timestamp into SQLite	`int()`, `float()`, `strftime`
table unchanged after UPDATE	forgot `commit()` (or WHERE matched nothing: `rowcount` 0)	`conn.commit()`; print `cur.rowcount`
`mat1 and mat2 shapes cannot be multiplied`	layer sizes do not chain	out of one = in of next
no error but `y` has shape `(N, N)` (or a loss warning)	`w` was `(F,)` not `(F,1)`: `(N,)` + noise `(N,1)` broadcasts silently	`w.reshape(F, 1)`; always print `X.shape, y.shape`
results differ every run	seed missing or set too late	`torch.manual_seed(42)` on the first line
`AttributeError: cannot assign module before Module.__init__()`	missing `super().__init__()`	add it first
`FileNotFoundError`	CSV not beside the script	check `os.getcwd()`

I. Last 10 minutes

- [ ] delete `.db`, run the **whole file** from a clean start - no errors
- [ ] every chart: title + both axis labels - every answer printed with a label
- [ ] shapes printed (`df.shape`, `X.shape`, `y.shape`); seed on the first torch line
- [ ] comments quote the question; names exact; file named as the paper says; saved and uploaded

T7 - PyTorch Basics: a tiny neural network, trained and plotted

Covers: tensors, `nn.Module`, `nn.Linear`, ReLU/Sigmoid, loss + optimiser, the 5-line training loop, loss recording and plotting, evaluation, DataLoader basics. Earns the 12 marks of 2024 "Python Task 3" (3a 4 + 3b 2 + 3c 3 + 3d 3). The mock exam may replace this task by a CSV/pandas/numpy/matplotlib task - see section 8.

1. What the exam asks

Only the **2024 paper** (and the course labs) use PyTorch. The 2021, 2022 and 2023 Python parts were tkinter + database tasks (see T8). The "Practice Exam Task 3" document is titled "*Practice Task 3 (replacing pytorch)*", which tells you the examiners may swap PyTorch for CSV/pandas/numpy/matplotlib.

2024 Final Lab Exam, Python Task 3: Basic Neural Network Implementation [12 marks]

> Define a class `SimpleNN` that extends `torch.nn.Module`. Input Layer: 10 input features. Hidden Layer 1: fully connected, 5 neurons, ReLU. Hidden Layer 2: fully connected, 3 neurons, ReLU. Output Layer: fully connected, 1 neuron, Sigmoid.

Part	Marks	Wording
3a	4	Instantiate `SimpleNN`. Use `torch.nn.MSELoss()` as loss. Use `torch.optim.SGD` with learning rate 0.01.
3b	2	Synthetic data: 100 samples, 10 features. Target is "a tensor with 100 values". Use `torch.randn` for input **and** target.
3c	3	Train 20 epochs. Each epoch: compute loss, backpropagate, update weights. **Record the training loss for each epoch.**
3d	3	Plot the training loss over epochs using matplotlib. "*Your plot should display the training loss decreasing over time, indicating that the model is learning.*"

Model answer in the course folder: `Practice/LabExam2024–25/ProgSD–Python–Solution.py` (Task 3a-3d). Course lab code: `WEEK2/tAPP_Lab03/Lab3_sol/pytorch/ex_2.py` - `ex_4.py` (MNIST) and `CIFAR.py`; notes in `pytorch.py`; extra practice in `PyTorch_neural_network_Practical exercises.docx` (RevenueNet regression with MSELoss + Adam; RiskNN binary classification with BCELoss + SGD).

1b. ADDED: the instructor's official PyTorch checklist (practical assessment)

The course instructor lists these PyTorch skills for the practical assessment, which "helps you prepare for the Python question in the final exam":

- Setting a random seed before generating data so that your results are reproducible each time you run your program.
- defining a simple neural network using `torch.nn.Module`
- using Linear layers and ReLU
- generating synthetic data (*Generate a continuous target:* `y = X @ w + b + noise`)
- understanding the shapes of X and y
- choosing the required loss function and optimiser
- writing a basic training loop
- recording and plotting the training loss

"Continuous target" with Linear + ReLU layers means a **regression** task: `MSELoss`, **no Sigmoid on the output**, and a loss that visibly falls. That differs from 2024 (random target, Sigmoid output), so **section 3.8 (new) is the main template to learn**; 3.1 stays valid for a 2024-style question. The instructor also says: *do not memorise one complete solution; practise the individual operations.* Section 3.8 is therefore split into seven small operations, and a complete runnable program is in `Practice/Solutions_py/pytorch_regression_practical.py`.

```
losses = [1.2, 1.1, 1.05, 1.0, 0.98]       # your list from the loop
plt.plot(range(1, len(losses) + 1), losses, marker="o")
plt.title("Training Loss Over Time")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.grid(True)
plt.show()
```

3.8 ADDED - REGRESSION: the instructor's task, step by step

Task shape: invent data with a **known rule** `y = X @ w + b + noise`, train a small network to rediscover that rule, record the loss every epoch, plot it. Seven independent operations; each block below runs on its own.

	2024 Task 3 (3.1)	Instructor-style regression (3.8)
Target	`torch.randn` noise (nothing to learn)	`X @ w + b + noise` (a real pattern)
Last layer	`Linear` then Sigmoid	`Linear` only, **no activation**
Loss	`MSELoss`	`MSELoss`
Loss curve	almost flat	falls steeply, then flattens at the noise floor

Analogy for the whole task: a shop with a secret price list. Each row of `X` is one customer's basket (how many of each of 4 items). `w` is the secret price of each item, `b` a fixed service fee, `noise` small rounding errors on the till. `y` is the final bill. The network never sees `w` and `b`; it sees only baskets and bills, and has to work the prices out.

3.8.1 Operation 1: the random seed (analogy: the recipe card for the shuffle)

A seed is a starting number for the random-number generator. Same seed = same "random" numbers in the same order, like shuffling a deck from a fixed starting order. Without it every run gives different data, different weights, different loss numbers.

```
import torch
import torch.nn as nn
import torch.optim as optim

# (a) the seed makes "random" numbers repeatable
torch.manual_seed(42)
a = torch.randn(3, 2)
torch.manual_seed(42)          # same seed again ...
b = torch.randn(3, 2)
torch.manual_seed(7)           # ... different seed
c = torch.randn(3, 2)
print(torch.equal(a, b), torch.equal(a, c))     # True False


# (b) the model weights are random too: seed BEFORE creating the model
def first_weight(set_seed):
    if set_seed:
        torch.manual_seed(42)
    layer = nn.Linear(4, 1)              # weights drawn at random here
    return layer.weight[0, 0].item()


print(first_weight(True) == first_weight(True))     # True
print(first_weight(False) == first_weight(False))   # False


# (c) the real proof: run the whole program twice, compare the loss lists
def run(seed):
    torch.manual_seed(seed)                              # FIRST line
    X = torch.randn(100, 4)
    w = torch.tensor([2.0, -1.0, 0.5, 3.0]).reshape(4, 1)
    y = X @ w + 1.5 + 0.1 * torch.randn(100, 1)
    model = nn.Sequential(nn.Linear(4, 16), nn.ReLU(), nn.Linear(16, 1))
    criterion = nn.MSELoss()
    optimizer = optim.Adam(model.parameters(), lr=0.05)
    losses = []
    for epoch in range(50):
        optimizer.zero_grad()
        loss = criterion(model(X), y)
```

```
        loss.backward()
        optimizer.step()
        losses.append(loss.item())
    return losses


print(run(42) == run(42))    # True: identical every run
print(run(42) == run(43))    # False: different seed, different run
```

Notes on the three parts:

- (a) After `manual_seed(42)`, `randn` gives the same values every time. A different seed gives different values.
- (b) **The model's starting weights are random too**, so the seed must come before `nn.Linear(...)` as well as before `torch.randn`. Put `torch.manual_seed(...)` right after the imports and you cover both.
- (c) Run the whole program twice with the same seed and compare the loss lists with `==`: `True` proves it is reproducible.
- Order matters: the *sequence* of random calls is what is fixed. If you create the model before the data, you still get repeatable results, just different numbers from creating them the other way round. Keep your order (seed, data, model) fixed.

3.8.2 Operation 2: make the data and understand the shapes (analogy: baskets, prices, bill)

```
import torch

torch.manual_seed(42)                       # 1. seed first
N, F = 100, 4                               # N rows (samples), F columns

X = torch.randn(N, F)                       # inputs, shape (100, 4)
w = torch.tensor([2.0, -1.0, 0.5, 3.0]).reshape(F, 1)   # (4, 1) COLUMN
b = 1.5                                     # bias: a single number
noise = 0.1 * torch.randn(N, 1)             # small randomness, (100, 1)
y = X @ w + b + noise                       # (100,4)@(4,1) -> (100,1)

print("X:", X.shape)                        # torch.Size([100, 4])
print("w:", w.shape)                        # torch.Size([4, 1])
print("y:", y.shape)                        # torch.Size([100, 1])
print((X @ w).shape, noise.shape)           # both (100, 1): they add up

# The classic mistake: w as a FLAT list of shape (4,)
w_flat = torch.tensor([2.0, -1.0, 0.5, 3.0])
print((X @ w_flat).shape)                   # torch.Size([100])  flat
print((X @ w_flat + b + noise).shape)       # torch.Size([100, 100])  WRONG
# Alternative that needs no reshape: random hidden weights
w2 = torch.randn(F, 1)
print((X @ w2).shape)                       # torch.Size([100, 1])
```

Shape arithmetic for `@` (matrix multiply: for each row of `X`, multiply by the matching entries of `w` and add up, i.e. one till receipt per row):

```
X       @     w      =     y_clean       + noise     -> y
(N, F)    (F, 1)        (N, 1)            (N, 1)       (N, 1)
(100,4)   (4,1)         (100,1)           (100,1)      (100,1)
 inner numbers must match (F = 4) ----^   outer numbers survive (N, 1)
```

- `X`: N rows (samples) by F columns (features). `y`: **one column**, N rows.
- `b` is one number; it is added to every row ("broadcasting").
- **TRAP:** if `w` has shape `(4,)` (no `.reshape(F, 1)`), then `X @ w` is a flat `(100,)` and adding the `(100, 1)` noise creates a `(100, 100)` table. No error, wrong data. Always print `y.shape`; it must be `(N, 1)`.
- Even if `y` accidentally comes out `(100,)`, `MSELoss(output, y)` with output `(100, 1)` only gives a `UserWarning` and a wrong loss (trap 5.3).

3.8.3 Operation 3: the model (analogy: an assembly line ending in a plain ruler)

```
import torch
import torch.nn as nn
```

```
class RegressionNet(nn.Module):
    def __init__(self, n_features, n_hidden):
        super(RegressionNet, self).__init__()
        self.fc1 = nn.Linear(n_features, n_hidden)   # F -> hidden
        self.fc2 = nn.Linear(n_hidden, 1)            # hidden -> 1 number

    def forward(self, x):
        x = torch.relu(self.fc1(x))    # hidden layer + ReLU
        x = self.fc2(x)                # last layer: raw number, NO activation
        return x


model = RegressionNet(4, 16)
print(model)
print(model(torch.randn(5, 4)).shape)      # torch.Size([5, 1])
print(sum(p.numel() for p in model.parameters()))   # 97 = (4*16+16)+(16+1)

# the same network as a one-liner (good for a quick test)
same = nn.Sequential(nn.Linear(4, 16), nn.ReLU(), nn.Linear(16, 1))
print(same(torch.randn(5, 4)).shape)       # torch.Size([5, 1])
```

- `Linear(F, hidden)`: mixes the F inputs into `hidden` new numbers. `ReLU`: one-way gate between layers (without it two Linear layers collapse into one big Linear and cannot bend). `Linear(hidden, 1)`: mixes down to **one number per sample**, the predicted bill.
- **No activation on the last layer.** The bill can be any size, positive or negative (here from about -6 to +8). A Sigmoid would be a ruler that stops at 1 (trap 5.15); a ReLU would be a ruler that cannot read below 0.
- 97 parameters = `4*16 + 16` (layer 1) + `16 + 1` (layer 2). The input size must equal your number of feature columns.
- Keep the **class** in the exam (the question says "extends `nn.Module`"); `nn.Sequential` is the quick version.

3.8.4 Operation 4: choose the loss and the optimiser (analogy: darts and stride length)

- **Loss =** `nn.MSELoss()` because the target is a continuous number. It squares each miss (so big misses hurt a lot) and averages: the "distance from the bullseye".
- **Optimiser** = who adjusts the weights; **learning rate** = the length of each stride downhill in fog. Too short and you barely arrive; too long and you leap over the valley or fly off.

```
import torch
import torch.nn as nn
import torch.optim as optim


def final_loss(optimiser_class, lr, epochs=200):
    torch.manual_seed(42)
    X = torch.randn(100, 4)
    w = torch.tensor([2.0, -1.0, 0.5, 3.0]).reshape(4, 1)
    y = X @ w + 1.5 + 0.1 * torch.randn(100, 1)
    model = nn.Sequential(nn.Linear(4, 16), nn.ReLU(), nn.Linear(16, 1))
    criterion = nn.MSELoss()                    # continuous target -> MSE
    optimizer = optimiser_class(model.parameters(), lr=lr)
    for epoch in range(epochs):
        optimizer.zero_grad()
        loss = criterion(model(X), y)
        loss.backward()
        optimizer.step()
    return loss.item()


for lr in (0.001, 0.01, 0.05, 0.1, 0.5):
    print(f"SGD  lr={lr:<6} final loss {final_loss(optim.SGD, lr):.4f}")
for lr in (0.001, 0.01, 0.05, 0.1):
    print(f"Adam lr={lr:<6} final loss {final_loss(optim.Adam, lr):.4f}")
```

Measured output (seed 42, 200 epochs, the start loss is 15.59):

```
SGD  lr=0.001  final loss 6.7254     <- too small: still far from done
SGD  lr=0.01   final loss 0.0986
```

```
SGD  lr=0.05   final loss 0.0318     <- good
SGD  lr=0.1    final loss 0.0194     <- good
SGD  lr=0.5    final loss nan        <- too big: weights explode
Adam lr=0.001  final loss 7.0802     <- too small
Adam lr=0.01   final loss 0.0296
Adam lr=0.05   final loss 0.0092     <- good, at the noise floor
Adam lr=0.1    final loss 0.0051     <- good
```

Rules of thumb: **Adam with lr 0.01-0.1** or **SGD with lr 0.05-0.1** for this task. If the question *names* the optimiser and lr, use exactly those. Symptoms: loss barely moves = lr too small (or too few epochs); loss `nan` or growing = lr too big.

3.8.5 Operation 5: the training loop with recording (analogy: rounds of quizzes in a score-book)

```python
import torch
import torch.nn as nn
import torch.optim as optim

torch.manual_seed(42)                                  # seed first
X = torch.randn(100, 4)
w = torch.tensor([2.0, -1.0, 0.5, 3.0]).reshape(4, 1)
y = X @ w + 1.5 + 0.1 * torch.randn(100, 1)            # noise std 0.1

model = nn.Sequential(nn.Linear(4, 16), nn.ReLU(), nn.Linear(16, 1))
criterion = nn.MSELoss()
optimizer = optim.Adam(model.parameters(), lr=0.05)

losses = []                               # the score-book
for epoch in range(200):
    optimizer.zero_grad()                 # 1. wipe the whiteboard
    predictions = model(X)                # 2. forward: sit the quiz
    loss = criterion(predictions, y)      # 3. score (points lost)
    loss.backward()                       # 4. teacher marks the paper
    optimizer.step()                      # 5. student adjusts habits
    losses.append(loss.item())            # record a plain float
    if epoch == 0 or (epoch + 1) % 40 == 0:
        print(f"Epoch {epoch + 1:03d} | Loss: {loss.item():.4f}")

print("noise floor about", round(0.1 ** 2, 2))   # 0.01 = std squared
```

Measured: the loss goes from **15.59** (epoch 1) to **0.0092** (epoch 200). The noise has standard deviation 0.1, so even a perfect model cannot get the MSE far below `0.1**2 = 0.01`; the network has reached that **noise floor**. (It may dip a little under it on the training data, because the network starts memorising the noise.)

Each line is the same as in section 4: `zero_grad` wipes the whiteboard, `model(X)` sits the quiz, `criterion` scores it, `backward` is the teacher marking which weights cost points, `step` is the student adjusting. `loss.item()` stores a plain float for plotting.

3.8.6 Operation 6: plot the loss curve

```python
import matplotlib.pyplot as plt

losses = [15.6 / (1 + 0.5 * i) ** 1.5 + 0.009 for i in range(200)]  # fake

plt.plot(range(1, len(losses) + 1), losses)
plt.title("Training Loss Over Time")
plt.xlabel("Epoch")
plt.ylabel("Loss (MSE)")
plt.grid(True)
plt.show()

# optional: the loss falls from 15 to 0.01, a log y-axis shows the tail
plt.figure()
plt.plot(range(1, len(losses) + 1), losses)
plt.yscale("log")
plt.title("Training Loss Over Time (log scale)")
plt.xlabel("Epoch")
plt.ylabel("Loss (MSE, log)")
plt.show()
```

Title, x label and y label are the marks. Put the `plt.show()` **after** the loop and plot `losses` (a list), not one number. The log version is optional but helpful: the first epochs (15 down to 1) dwarf the final 0.01 on a linear axis.

3.8.7 Operation 7: evaluate (analogy: the real exam hall) and the sanity check

(a) Predictions vs true values, with `model.eval()` and `torch.no_grad()`:

```
import torch
import torch.nn as nn
import torch.optim as optim

torch.manual_seed(42)
X = torch.randn(100, 4)
w = torch.tensor([2.0, -1.0, 0.5, 3.0]).reshape(4, 1)
y = X @ w + 1.5 + 0.1 * torch.randn(100, 1)
model = nn.Sequential(nn.Linear(4, 16), nn.ReLU(), nn.Linear(16, 1))
criterion = nn.MSELoss()
optimizer = optim.Adam(model.parameters(), lr=0.05)
for epoch in range(200):
    optimizer.zero_grad()
    loss = criterion(model(X), y)
    loss.backward()
    optimizer.step()

model.eval()                              # exam-hall mode
with torch.no_grad():                     # no gradients, no learning
    predictions = model(X)
print("predicted   true")
for i in range(5):                        # first five samples
    print(f"{predictions[i].item():9.3f} {y[i].item():9.3f}")
errors = (predictions - y).abs()
print("mean absolute error:", round(errors.mean().item(), 3))
```

Measured: the five predictions are within about 0.05 of the truth (e.g. `-2.026` vs `-1.997`), mean absolute error 0.077 over all 100.

(b) Train / test split (80 / 20). Make *all* data after the seed, then slice. Learn from the first 80 rows only, then measure the loss on the last 20 rows the model has never seen:

```
import torch
import torch.nn as nn
import torch.optim as optim

torch.manual_seed(42)                     # seed, then make ALL the data
X = torch.randn(100, 4)
w = torch.tensor([2.0, -1.0, 0.5, 3.0]).reshape(4, 1)
y = X @ w + 1.5 + 0.1 * torch.randn(100, 1)

X_train, X_test = X[:80], X[80:]          # first 80 rows: practice
y_train, y_test = y[:80], y[80:]          # last 20 rows: unseen exam

model = nn.Sequential(nn.Linear(4, 16), nn.ReLU(), nn.Linear(16, 1))
criterion = nn.MSELoss()
optimizer = optim.Adam(model.parameters(), lr=0.05)
for epoch in range(200):                  # learn from the TRAIN part only
    optimizer.zero_grad()
    train_loss = criterion(model(X_train), y_train)
    train_loss.backward()
    optimizer.step()

model.eval()
with torch.no_grad():                     # measure on the TEST part
    test_loss = criterion(model(X_test), y_test)
print(f"train MSE {train_loss.item():.4f} | test MSE {test_loss.item():.4f}")
```

Measured: `train MSE 0.0065 | test MSE 0.0636`. The test loss is higher: the 16-neuron network has partly memorised 80 noisy rows. That gap (train much lower than test) is called *overfitting*; a good report sentence is "the model generalises reasonably well, test error is small compared with the target's range, roughly -6 to +8".

(c) Sanity check: a single `Linear(F, 1)` layer must recover the secret prices. A linear model has exactly the form `X @ w + b`, so training it on this data should *discover* `w` and `b`:

```
import torch
import torch.nn as nn
import torch.optim as optim

torch.manual_seed(42)
X = torch.randn(100, 4)
w = torch.tensor([2.0, -1.0, 0.5, 3.0]).reshape(4, 1)
y = X @ w + 1.5 + 0.1 * torch.randn(100, 1)

model = nn.Linear(4, 1)                    # NO hidden layer, no ReLU
criterion = nn.MSELoss()
optimizer = optim.SGD(model.parameters(), lr=0.1)
for epoch in range(200):
    optimizer.zero_grad()
    loss = criterion(model(X), y)
    loss.backward()
    optimizer.step()

print("learned w:", [round(v, 3) for v in model.weight.flatten().tolist()])
print("true    w:", w.flatten().tolist())
print("learned b:", round(model.bias.item(), 3), " true b: 1.5")
print("final loss:", round(loss.item(), 4))
```

Measured: learned `[1.992, -1.004, 0.492, 3.0]` against true `[2.0, -1.0, 0.5, 3.0]`, bias `1.504` against `1.5`. If your learned numbers match the secret ones, your data, loss and loop are all correct. It is the best "does my pipeline work" check.

The complete program (seed, data, model, train, record, plot, evaluate, sanity check, reproducibility proof) with heavy comments: `Practice/Solutions_py/pytorch_regression_practical.py`.

5.11 ADDED - Seed in the wrong place (results differ every run) No error; two runs print different losses. The seed must run **before** `torch.randn(...)` **and** before `nn.Linear(...)` / `SimpleNN()` (weights are random too). In 3.8.1 (b): `first_weight(False)` gives a different weight each call. Fix: `torch.manual_seed(42)` on the line right after the imports, executed once per run.

5.12 ADDED - `w` **has shape** `(F,)` **instead of** `(F, 1)` No error. `X @ w` is flat `(100,)`; adding noise `(100, 1)` broadcasts to `(100, 100)` (3.8.2). Fix: `torch.tensor([...]).reshape(F, 1)` or `torch.randn(F, 1)`, then print `y.shape` and see `(100, 1)`.

5.13 ADDED - `y` **is** `(N,)` **but the model output is** `(N, 1)` Only a `UserWarning` from `MSELoss`, and a wrong, too-flat loss. Fix: build `noise` as `torch.randn(N, 1)`, or `y.reshape(-1, 1)` / `y.unsqueeze(1)`. See also 5.3.

5.14 ADDED - Forgetting `optimizer.zero_grad()` **in a regression loop** Measured (Adam lr 0.05, same data): with `zero_grad` the loss at epoch 200 is 0.0092; **without** it the loss went 15.59, 2.80 (epoch 10), then **444.5** (epoch 100) and 202.7 (epoch 200). Symptom: the loss falls at first, then bounces up and never settles. Fix: `optimizer.zero_grad()` as the first line of the loop.

5.15 ADDED - Sigmoid (or ReLU) on the last layer of a regression model Measured: Sigmoid on the output, same data and settings: the loss stays stuck at **10.12** and never goes near 0.01. The bills range from -6.1 to +8.3 but a Sigmoid can only output numbers between 0 and 1. Fix: last layer is a plain `nn.Linear(hidden, 1)`.

5.16 ADDED - Learning rate wrong Loss barely moves (e.g. 15.59 to 14.43 after 200 epochs with SGD lr 0.0001): lr too small. Loss becomes `nan` (SGD lr 0.5 in 3.8.4): lr too big. Try `0.01 / 0.05 / 0.1`.

6. Anticipated exam questions

Q1 [4] Define `SimpleNN` with layers 10 -> 5 (ReLU) -> 3 (ReLU) -> 1 (Sigmoid). Answer: class in 3.1: three `nn.Linear` in `__init__` (after `super().__init__()`), and `torch.relu` / `torch.relu` / `torch.sigmoid` in `forward`, returning `x`.

Q2 [2] Create the model, `MSELoss` and SGD with lr 0.01.

```
model = SimpleNN()
criterion = nn.MSELoss()
optimizer = optim.SGD(model.parameters(), lr=0.01)
```

Q3 [2] Generate 100 samples with 10 features and 100 target values using `torch.randn`.

```
data = torch.randn(100, 10)
target = torch.randn(100, 1)
```

Q4 [3] Train for 20 epochs and record each epoch's loss. Loop in 3.1: `zero_grad, forward, criterion, backward, step`, then `losses.append(loss.item())`.

Q5 [3] Plot the loss against epoch with labels and title. 3.7: `plt.plot(...)`, `title`, `xlabel`, `ylabel`, `plt.show()`.

Q6 [2] Your loss only falls from 1.1955 to 1.1912. Is the model learning? Why so little? Yes, it is moving in the right direction. Targets are random noise (nothing to learn), only 20 full-batch updates at lr 0.01 are made, and the sigmoid output cannot reach negative targets. A learnable target, Adam or mini-batches show a larger drop (3.4, 3.5).

Q7 [2] What does `optimizer.zero_grad()` do and why is it needed? It resets the stored gradients to zero. PyTorch *adds* new gradients to old ones on every `backward()`, so without it each update would use stale, accumulated feedback.

Q8 [2] Why `loss.item()`? It converts the one-value loss tensor to a normal Python float, so the list can be plotted and does not keep the autograd graph alive.

Q9 [3] Evaluate on unseen data. Split data (e.g. 80/20), train on the first part, then `model.eval()` and `with torch.no_grad():` compute outputs/accuracy on the second part (3.6).

Q10 [2] Which loss for (a) predicting house price, (b) spam yes/no, (c) 10 digit classes? (a) `MSELoss`; (b) `BCELoss` with sigmoid output; (c) `CrossEntropyLoss` with 10 raw outputs and integer labels.

Q11 ADDED [2] Why do we set `torch.manual_seed(...)` and where does it go? So the random data and the random starting weights are the same on every run, making results reproducible and comparable. It goes at the top, before `torch.randn(...)` and before the model is created (3.8.1).

Q12 ADDED [4] Generate 100 samples with 4 features and a continuous target `y = X @ w + b + noise`. State the shapes.

```
torch.manual_seed(42)
X = torch.randn(100, 4)                                  # (100, 4)
w = torch.tensor([2.0, -1.0, 0.5, 3.0]).reshape(4, 1)    # (4, 1)
y = X @ w + 1.5 + 0.1 * torch.randn(100, 1)              # (100, 1)
```

`X @ w` is matrix multiplication: (100,4) times (4,1) gives (100,1). `b` (1.5) is added to every row.

Q13 ADDED [3] Define a network with one hidden layer of 16 neurons for 4 inputs, predicting one continuous value. Which loss and which last-layer activation? `Linear(4,16)`, ReLU, `Linear(16,1)`; **no** activation on the output; `nn.MSELoss()` (3.8.3, 3.8.4).

Q14 ADDED [3] Choose and justify an optimiser and learning rate. `optim.Adam(model.parameters(), lr=0.05)` (or SGD with lr 0.05-0.1). It adapts well, reached the noise floor (0.0092) in 200 epochs. Too small an lr (0.001) is still at about 7 after 200 epochs; too large (SGD 0.5) gives `nan`.

Q15 ADDED [2] The loss ends near 0.01. Is that good? Yes. The noise has standard deviation 0.1, so the lowest achievable MSE is about 0.1 squared = 0.01: the model has learned the pattern and what is left is the unavoidable noise.

Q16 ADDED [3] How can you check that the model really learned the rule? Compare predictions with `y` for a few samples under `no_grad`; evaluate on held-out rows (train/test split); or train a single `nn.Linear(4, 1)` and print `model.weight` and `model.bias`, which should be close to `w` and `b` (3.8.7).

Q17 ADDED [2] Training loss is 0.0065 but test loss is 0.0636. What does it mean? The model fits the training rows better than unseen rows (overfitting): it has partly memorised the noise. Remedies: more data, a smaller network, or fewer epochs.

7. Quick-reference box (copy onto the notes sheet)

```
import torch, torch.nn as nn, torch.optim as optim
import matplotlib.pyplot as plt
class Net(nn.Module):
  def __init__(self):
    super(Net, self).__init__()       # FIRST LINE, always
    self.fc1 = nn.Linear(10, 5)       # spec "10,5,3,1": pairs of numbers
    self.fc2 = nn.Linear(5, 3); self.fc3 = nn.Linear(3, 1)
  def forward(self, x):               # name must be forward; return x
    x = torch.relu(self.fc1(x)); x = torch.relu(self.fc2(x))
    return torch.sigmoid(self.fc3(x))
model = Net(); criterion = nn.MSELoss()   # BCELoss / CrossEntropyLoss
optimizer = optim.SGD(model.parameters(), lr=0.01)  # or optim.Adam
data = torch.randn(100, 10); target = torch.randn(100, 1)  # (100,1) !
losses = []
for epoch in range(20):
  optimizer.zero_grad(); out = model(data)   # wipe -> forward
  loss = criterion(out, target)              # score
  loss.backward(); optimizer.step()          # mark -> adjust
  losses.append(loss.item())                 # .item() = float
plt.plot(losses); plt.title("..."); plt.xlabel("Epoch")
plt.ylabel("Loss"); plt.show()
EVAL: model.eval(); with torch.no_grad(): out = model(test_x)
 binary: (out > 0.5).float() | multi: _, pred = torch.max(out, 1)
BATCHES: DataLoader(TensorDataset(X, y), batch_size=10, shuffle=True)
 for xb, yb in loader: (same 5 lines)   len(loader) = no. of batches
PAIRS: MSE -> float same shape | BCE -> sigmoid + float | CE -> raw + long
Flat loss with randn target = EXPECTED. torch.manual_seed(42) for repeats.
```

7 (continued) ADDED - regression extras for the notes sheet

```
SEED first (before randn AND before the model): torch.manual_seed(42)
X = torch.randn(N, F)                          # (N, F)
w = torch.tensor([2., -1., .5, 3.]).reshape(F, 1)   # COLUMN (F, 1)
y = X @ w + b + 0.1 * torch.randn(N, 1)        # (N,F)@(F,1) -> (N,1)
print(X.shape, y.shape)                        # check! y is (N, 1)
model: Linear(F,16) -> ReLU -> Linear(16,1)    # NO sigmoid on the output
loss = nn.MSELoss(); Adam lr 0.01-0.1 | SGD lr 0.05-0.1; 100-200 epochs
loop: zero_grad, forward, loss, backward, step, losses.append(loss.item())
noise floor: MSE ~ noise_std**2 (0.1 -> 0.01). lr too big: nan.
check: model.eval(); with torch.no_grad(): pred = model(X[80:])
 sanity: single nn.Linear(F,1) recovers w, b via model.weight, model.bias
 reproducible: run twice, losses1 == losses2 -> True
```

8. If PyTorch is replaced

The *Practice Exam Task 3* document is headed "replacing pytorch": a CSV file read and written with pandas/numpy, summary statistics saved to a text file, and a matplotlib histogram. If the mock exam does this, nothing in this document is needed except the **plotting habit** (3.7: title, both axis labels, `plt.show()`) and the general loop/list/function idioms. Go instead to **T3, T5 and T6** (the pandas/CSV, numpy and matplotlib topic documents in this folder; check the pack index for exact titles) and the solution documents for the full worked tasks. A sensible split of last-minute revision time: T3/T5/T6 first, SQLite second, this document third, T8 (tkinter) never.